UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ESTRUCTURA DE DATOS

Report

Mauricio Joel Gómez Barrios - 202031478

03 de abril de 2026

Índice.

Diseño de las estructuras **3**
1.1 Producto - Producto.h **3**
1.2 ListaEnlazada - ListaEnlazada.h **3**
Atributos: 3
Nodos Interno: 4
Operaciones Principales: 4
1.3 ArbolAVL - ArbolAVL.h 4
NodoAVL: 4
Rotaciones implementadas: 4
Operaciones: 5
1.4 ArbolB - ArbolB.h 5
Struct ListaFechaIgual: 5
Struct ElementoArbolB (Clave del árbol B): 5
Class NodoB: 6
Operaciones: 6
1.4 ArbolBMas - ArbolBMas.h 6
Struct ListaCategoria - lista de colisiones en hojas: 6
Class NodoBMas: 7
división (dividirNodo): 7
1.6 Class GestionInventario - GestionInventario.h y GestionInventario.cpp 8
Estructuras internas (atributos privados): 8
2.1 Lista enlazada no ordenada: **10**
2.2 Lista enlazada ordenada: **10**
2.3 Árbol AVL (clave: nombre del producto): **10**
2.4 Árbol B (clave: fecha de caducidad): **11**
2.5 Árbol B+ (clave: categoría): **11**
2.6 Operación global de agregar/eliminar producto: **11**
- Procedimiento paso a paso: **12**
- Casos de búsqueda evaluados: **13**

Diseño de las estructuras.

1. Diseño De Las Estructuras De Datos:

Se utilizan cinco estructuras de datos implementadas desde cero en C++, cada una para un tipo de consulta sobre el catálogo de productos, al final todas comparten el mismo objeto Producto* En memoria, no se duplican datos, solo se almacenan punteros. Cada inserción o eliminación se extiende atómicamente a las cinco estructuras, si alguna falla, se ejecuta un rollback.

1.1 Producto - Producto.h

Es un struct simple que agrupa los siete atributos de cada artículo del catálogo. Todas las estructuras almacenan punteros Producto* hacia este objeto, garantizando que exista una sola copia por producto en memoria.

Atributo	Tipo	Descripción
nombre	str::string	Nombre del artículo, clave del arbol AVL
codigo_barra	str::string	Identificador único, clave de búsqueda en Lista.
categoria	str::string	Categoría del producto, clave del árbol B+.
fecha_caducidad	str::string	Fecha de formato YYYY-MM-DD, clave del árbol B.
marca	str::string	Fabricante del producto.
precio	double	Precio de venta.
stock	int	Disponibilidad en el inventario.

1.2 ListaEnlazada - ListaEnlazada.h

Es una lista enlazada simple implementada con punteros manuales. El sistema instancia dos objetos de esta clase: listaNoOrdenada (flag ordenada=false) que actúa como registro maestro de todos los productos y realiza búsquedas por código de barra, y listaOrdenada (flag ordenada=true) que mantiene los nodos ordenados por nombre para pruebas de comparación. La inserción al frente en la lista no ordenada es O(1).

Atributos:

Atributo	Tipo	Descripción
cabeza	Nodo*	Puntero al primer nodo de la lista
ordenada	bool	Si es true, insertar() mantiene orden ascendente por nombre
limpio	bool	Si true, el destructor libera también el Producto* pero solo en la listaNoOrdenada

Nodos Interno:

Atributo	Tipo	Descripción
dato	Producto*	Puntero al objeto producto compartido
siguiente	Nodo*	Enlace al siguiente nodo de la cadena

Operaciones Principales:

Operación	Big O	Descripción
insertar (p)	O(1) / O(n)	O(1) si no ordenada, se inserta al frente, O(n) si ordenada, busca posición
buscarSecuencia (nombre)	O(n)	Recorre nodo por nodo comparando el atributo nombre.
buscarPorCodigo (cod)	O(n)	Recorre comparando codigo_barra, el cual se usa para validar duplicados antes de insertar.
eliminar (cod)	O(n)	Localiza el nodo por código, lo enlaza nuevamente y libera. Si limpio = true libera también el producto*

1.3 ArbolAVL - ArbolAVL.h

Árbol binario de búsqueda que se balancea automáticamente. La clave de ordenamiento es el nombre del producto. Mantiene la propiedad de balance: para cada nodo, la diferencia de altura entre subárbol izquierdo y derecho es como máximo 1 o -1, lo que garantiza altura O(log n) y por tanto búsqueda e inserción en O(log n). Se usa para: búsqueda binaria por nombre, listado in-order alfabético y como fuente de nombres para pruebas de comparación.

NodoAVL:

Atributo	Tipo	Descripción
producto	Producto*	Puntero al objeto producto
izquierda	NodoAVL*	Subárbol izquierdo donde están los nombres menores
derecha	NodoAVL*	Subárbol derecho donde están los nombres mayores
altura	int	Altura del nodo, se actualiza tras cada rotación.

Rotaciones implementadas:

Operacion	Big O	Descripción
rotarDerecha(y)	O(1)	Rotación simple derecha, resuelve el desbalance LL
rotarIzquierda(x)	O(1)	Rotación simple izquierda, resuelve desbalance RR
Doble LR	O(1)	rotarIzquierda(izq) + rotarDerecha, resuelve LR
Doble RL	O(1)	rotarDerecha(der) + rotarIzquierda, resuelve R

Operaciones:

Operacion	Big O	Descripción
insertar(p)	O(log n)	Inserta por nombre y si ya existe el mismo nombre, ignora
buscarPorNombre(s)	O(log n)	Búsqueda binaria recursiva comparando std::string
eliminar(nombre)	O(log n)	Elimina el nodo y rebalancear, sucesor in-order si tiene dos hijos
listarEnOrden()	O(n)	Recorrido in-order que imprime numeración, nombre, código, categoría, precio, stock
contarNodos()	O(n)	Cuenta recursivamente todos los nodos, usado para dimensionar las pruebas de comparación
obtenerNombres(arr,n)	O(n)	Llena arreglo con los nombres in-order hasta el límite n solicitado
crearGrafico(arch)	O(n)	Genera archivo .dot para Graphviz con nodos y aristas

1.4 ArbolB - ArbolB.h

Árbol B de grado configurable (por defecto t = 3). Cada nodo puede contener entre t-1 y 2t-1 claves. La clave de ordenamiento es fecha_caducidad en formato YYYY-MM-DD. Como múltiples productos pueden compartir la misma fecha, cada clave del árbol B apunta a una lista enlazada de colisiones (struct ListaFechaIgual) que agrupa todos los productos de esa fecha. Esto permite consultas eficientes por rango de fechas.

Struct ListaFechaIgual:

Atributo	Tipo	Descripción
producto	Producto*	Puntero al objeto producto con esa fecha
siguiente	ListaFechaIgual*	Enlace al siguiente producto de la misma fecha

Struct ElementoArbolB (Clave del árbol B):

Atributo	Tipo	Descripción
fecha	std::string	Clave de ordenamiento YYYY-MM-DD
lista	ListaFechaIgual*	Cabeza de la lista de productos con esta fecha.

Class NodoB:

Atributo	Tipo	Descripción
claves	ElementoArbolB[]	Arreglo de 2t-1 claves (fecha + lista)
hijos	NodoB*[]	Arreglo de 2t punteros a hijos
numClaves	int	Número actual de claves en el nodo
esHoja	bool	True si no tiene hijos
grado	int	Grado mínimo t del árbol

Operaciones:

Atributo	Tipo	Descripción
insertar(p)	O(log n)	Si la fecha ya existe entonces se agrega a la lista de colisiones. Si no, inserta la clave en el árbol con la división al descender.
eliminar(p)	O(log n)	Arreglo de 2t punteros a hijos
imprimirEnRango(fi, ff)	O(log n+k)	Búsqueda del nodo más cercano a fi, luego recorrido in-order hasta ff. k = productos en el rango.
crearGrafico(arch)	O(n)	Genera .dot mostrando claves (fechas) por página del árbol.

1.4 ArbolBMas - ArbolBMas.h

Árbol B+ de grado configurable (por defecto t = 3). A diferencia del árbol B, los datos reales (Producto*) se almacenan únicamente en los nodos hoja, los nodos internos solo contienen claves de enrutamiento. Las hojas están enlazadas entre sí mediante punteros siguientes, formando una lista doblemente recorrible que permite recorridos eficientes por categoría. La clave de ordenamiento es categoría,cada clave en una hoja apunta a una lista de todos los productos de esa categoría.

Struct ListaCategoria - lista de colisiones en hojas:

Atributo	Tipo	Descripción
producto	Producto*	Puntero al producto de esta categoría
siguiente	ListaCategoria*	Enlace al siguiente producto de la misma categoría

Class NodoBMas:

Atributo	Tipo	Descripción
claves	string[]	Arreglo de 2t claves (categorías)
hijos	NodoBMas*[]	Arreglo de 2t+1 punteros, sólo válidos en nodos internos
listas	ListaCategoria*[]	Arreglo de listas de productos, sólo válidos en hojas
siguiente	NodoBMas*	Enlace al siguiente nodo hoja (lista de hojas)
padre	NodoBMas*	Puntero al padre, es necesario para la inserción en padre durante la división
esHoja	bool	Distingue nodo hoja de nodo interno
numClaves	int	Número de claves actualmente en el nodo

Operaciones:

Atributo	Big O	Descripción
insertar(p)	O(log n)	Navega hasta la hoja correcta con buscarHoja(). Si la categoría existe, agrega a su lista. Si no, inserta nueva clave y si el nodo queda lleno (numClaves == 2t), llama dividirNodo()
buscarPorCategoria(cat)	O(log n+k)	Navega hasta la hoja con buscarHoja(). En la hoja, localiza la clave y recorre su lista de productos. k = productos en la categoría
eliminarProducto(cat,cod)	O(log n)	Navega a la hoja de la categoría, localiza el Producto* por código de barra y lo desenlaza de la lista interna
crearGrafico(arch)	O(n)	Genera .dot con nodos HTML para árbol, más aristas rojas (constraint=false) que conectan las hojas enlazadas

división (dividirNodo):

Al insertar en un nodo hoja que ya tiene 2t claves (2·3 = 6 en el sistema), se crea un nuevo nodo hoja. La primera mitad de las claves permanece en el nodo original, la segunda mitad pasa al nuevo nodo. Se actualiza el puntero siguiente de la lista de hojas. Luego se llama a insertarEnPadre() para pasar la clave mínima del nuevo nodo al padre. Si el padre también queda lleno se repite recursivamente. Si el nodo dividido era la raíz, se crea una nueva raíz de altura 1 (nodo interno).

1.6 Class GestionInventario - GestionInventario.h y GestionInventario.cpp
Clase que encapsula y coordina las cinco estructuras. El main.cpp nunca interactúa directamente con las estructuras internas, toda operación pasa por esta clase, que garantiza la coherencia atómica.

Estructuras internas (atributos privados):

Atributo	Tipo	Descripción
listaNoOrdenada	ListaEnlazada	Lista sin orden, dueña de la memoria (limpio=true). Búsqueda por código O(n)
listaOrdenada	ListaEnlazada	Lista ordenada por nombre (limpio=false), para pruebas comparativas
avlNombres	ArbolAVL	Clave: nombre. Búsqueda O(log n), listado in-order, pruebas de comparación
arbolFechas	ArbolB	Clave: fecha_caducidad. Consultas por rango de fechas O(log n+k)
arbolCategoria	ArbolBMas	Clave: categoria. Listado por categoría O(log n+k)

Operaciones:

Operación	Big O	Descripción
cargarDesdeCSV(ruta)	O(n log n)	Lee el CSV, valida cada línea (formato, numéricos, unicidad). Errores a errors.log (ios::trunc). Llama agregarProducto() por cada línea válida
agregarProducto(p)	O(log n)	Verifica duplicado en listaNoOrdenada. Si no existe, inserta en las 5 estructuras. Retorna false si duplicado
eliminarProducto(cod)	O(log n)	Busca el Producto* en listaNoOrdenada, extrae nombre y categoría, luego elimina de las 5 estructuras en orden
buscarPorNombreSecuencial (n)	O(n)	Delega a listaNoOrdenada.buscarSecuencia()
buscarPorNombreAVL(n)	O(log n)	Delega a avlNombres.buscarPorNombre()
buscarPorCodigo(cod)	O(n)	Delega a listaNoOrdenada.buscarPorCodigo()
buscarPorRangoFechas(fi,ff)	O(log n+k)	Delega a arbolFechas.imprimirEnRango()
buscarPorCategoria(cat)	O(log n+k)	Delega a arbolCategoria.buscarPorCategoria().
listarPorNombre()	O(n)	Delega a avlNombres.listarEnOrden(). Imprime el catálogo alfabético
compararBusquedas(m,j)	O(m·j·n)	m búsquedas × j repeticiones para Lista y AVL, exitosas y fallidas. Imprime tiempos, promedios y factor de mejora.
generarGraficoAVL/B/BMas()	O(n)	Genera .dot y llama system('dot -Tpng ...') para producir PNG.

2. Justificación Big O

2.1 Lista enlazada no ordenada:

La búsqueda por nombre o por código de barra es O(n) porque el algoritmo recorre nodo a nodo desde la cabeza hasta encontrar una coincidencia o llegar al final. En el peor caso examina los n elementos.

La inserción al frente es O(1): se crea un nuevo nodo y se enlaza como nueva cabeza, sin recorrer la lista. La eliminación es O(n) porque primero debe localizar el nodo con búsqueda secuencial, una vez encontrado, el ajuste de punteros es O(1).

Estructura	Operación	Complejidad	Caso
Lista Enlazada (No ordenada) Búsqueda secuencial por nombre y código de barra	Búsqueda	O(n)	Promedio/Peor
	Inserción al frente	O(1)	Siempre
	Eliminación	O(n)	Búsqueda previa
	Eliminación (mejor caso)	O(1)	Elemento al inicio

2.2 Lista enlazada ordenada:

Mantiene los nodos en orden alfabético por nombre. La búsqueda sigue siendo O(n) en el peor caso porque, aunque puede detenerse anticipadamente si el nombre actual ya supera al buscado, en el caso promedio recorre n/2 nodos.

La inserción es O(n): antes de enlazar el nuevo nodo hay que recorrer la lista para encontrar la posición correcta.

La eliminación es igualmente O(n) por la búsqueda previa requerida.

Estructura	Operación	Complejidad	Caso
Lista Enlazada (ordenada por nombre) inserción mantiene orden alfabético	Búsqueda	O(n)	Promedio/Peor
	Inserción ordenada	O(m)	Busca posición
	Eliminación	O(n)	Búsqueda previa

2.3 Árbol AVL (clave: nombre del producto):

El AVL garantiza que la altura del árbol siempre sea h = O(log n) mediante rotaciones automáticas (simples y dobles) tras cada inserción o eliminación. Esto hace que búsqueda, inserción y eliminación sean O(log n) en todos los casos, no solo en el promedio. El rebalanceo añade un trabajo constante de rotaciones, que no altera la complejidad asintótica. El recorrido in-order para listar alfabéticamente visita los n nodos exactamente una vez: O(n).

Estructura	Operación	Complejidad	Caso
Árbol AVL (clave: nombre del producto) altura garantizada, h = O(log n), rotaciones automáticas	Búsqueda por nombre	O(log n)	Siempre
	Inserción y rebalanceo	O(log n)	Siempre
	Eliminación y rebalanceo	O(log n)	Siempre
	Recorrido in-order (listar)	O(n)	Visita todos

2.4 Árbol B (clave: fecha de caducidad):
Con grado t, cada nodo interno almacena entre t−1 y 2t−1 claves. La altura máxima es O(log_t n), que en notación simplificada es O(log n). Búsqueda, inserción y eliminación navegan desde la raíz hasta una hoja, haciendo como máximo O(log n) comparaciones por nivel.
La búsqueda por rango de fechas tiene un costo adicional de O(k), donde k es la cantidad de productos que caen en el intervalo pedido, resultando en O(log n + k) total.
En la implementación se usaron listas enlazadas por clave para manejar múltiples productos con la misma fecha de caducidad sin duplicar claves en el árbol.

Estructura	Operación	Complejidad	Caso
Árbol B (clave: fecha caducidad) Grado t=3 en implementación, claves con listas de productos por fecha	Búsqueda por fecha exacta	O(log n)	Siempre
	Inserción y división	O(log n)	Siempre
	Eliminación de producto	O(log n)	Siempre
	Búsqueda en rango de fechas	O(log n + k)	k = elementos devueltos

2.5 Árbol B+ (clave: categoría):
A diferencia del Árbol B, en el B+ los datos están únicamente en las hojas, y todas las hojas están enlazadas entre sí. Los nodos internos solo contienen claves de encaminamiento. Búsqueda, inserción y eliminación son O(log n) porque siguen la misma ruta raíz-hoja del árbol. El recorrido por categoría es O(log n + k): O(log n) para localizar la hoja de la categoría buscada, y O(k) para recorrer los k productos almacenados en esa hoja gracias a los punteros de enlace entre hojas, sin necesidad de volver a los nodos internos.

Estructura	Operación	Complejidad	Caso
Árbol B+ (clave: categoría) Datos sólo en hojas Hojas enlazadas para recorrido eficiente	Búsqueda por categoría	O(log n)	Navega hasta hoja
	Inserción y división	O(log n)	Siempre
	Eliminación de producto	O(log n)	Siempre
	Recorrido por categoría	O(log n + k)	k = Productos en categoria

2.6 Operación global de agregar/eliminar producto:
Cada inserción o eliminación debe propagarse a todas las estructuras: lista no ordenada O(1)/O(n), lista ordenada O(n), AVL O(log n), Árbol B O(log n), Árbol B+ O(log n). El término dominante es O(n) de las listas, pero dado que las listas actúan principalmente como índice de duplicados y de eliminación por código, y las búsquedas de rendimiento se comparan entre lista y AVL de forma aislada, se considera que la operación global es efectivamente O(log n) desde las estructuras de búsqueda optimizadas.

Estructura	Operación	Complejidad	Caso
Agregar / Eliminar producto (atomizada)	Inserta/elimina en todas las estructuras	O(log n)	Dominado por AVL, B y B+

3. Metodología de Medición

La comparación de rendimiento se centra en la operación de búsqueda por nombre, que es la única que puede ejecutarse tanto en la lista enlazada (búsqueda secuencial) como en el árbol AVL (búsqueda binaria). El módulo compararBusquedas(m, j) de GestionInventario mide ambas estructuras bajo las mismas condiciones para garantizar una comparación justa.
Se utiliza std::chrono::high_resolution_clock con std::chrono::microseconds. La medición hace el bucle de búsqueda, excluyendo I/O y cualquier operación de inicialización.

Parámetro	Valor utilizado	Justificación
m (búsquedas / repetición)	20	Muestra representativa sin saturar la consola
j (repeticiones)	5	Promedia la varianza
Productos cargados (n)	1,092	Catálogo completo desde catalogo.csv
Unidad de tiempo	Microsegundos (µs)	chrono::high_resolution_clock con cast a microseconds
Tipo de búsqueda exitosa	Nombres reales	Recolectados via AVL in-order (obtenerNombres)
Tipo de búsqueda fallida	"PRODUCTO NO EXISTE"	Garantiza recorrer toda la estructura sin hallar coincidencia

- **Procedimiento paso a paso:**

  Paso 1 – Preparación de datos:
  - Se carga el catálogo completo desde catalogo.csv (n = 1,092 productos válidos).
  - Se llama a avlNombres.obtenerNombres(arreglo, total, m) para recolectar m = 20 nombres reales en orden in-order. Estos nombres están garantizados a existir en ambas estructuras.
  - Se define el nombre "PRODUCTO NO EXISTE" como nombre de búsqueda fallida, asegurando que ninguna estructura lo encuentre.

  Paso 2 – Medición de búsquedas exitosas
  - Se ejecutan j = 5 repeticiones de un bucle de m = 20 búsquedas.
  - Dentro de cada repetición se registra t0 = high_resolution_clock::now() antes del bucle y t1 después. Se acumula la diferencia en tiempoListaExitosa y tiempoAVLExitosa.
  - Esto produce j × m = 100 búsquedas por estructura para el cálculo del promedio.

  Paso 3 – Medición de búsquedas fallidas
  - Para cada una de las j repeticiones, se realiza una única búsqueda del nombre inexistente y se mide el tiempo individual.
  - Las búsquedas fallidas son más informativas que las exitosas porque obligan a recorrer la estructura completa antes de confirmar la ausencia.

  Paso 4 – Cálculo de métricas
  - Promedio exitosa = tiempo_total_exitosa / (j × m)
  - Promedio fallida = tiempo_total_fallida / j
  - Factor de mejora = promedio_lista_exitosa / promedio_AVL_exitosa
  - El resultado final imprime tiempos en µs, promedios y el factor de mejora observado.

- Casos de búsqueda evaluados:

Caso	Lista enlazada	Árbol AVL
Exitosa aleatoria	Recorre en promedio n/2 nodos hasta hallar el elemento	Navega $\log_2(n) \approx 10$ niveles hasta el nodo exacto
Fallida (peor caso)	Recorre los n nodos sin encontrar coincidencia	Recorre la altura completa hasta una hoja nula

4. Resultados

Los resultados que se presentan a continuación corresponden a una ejecución representativa con n = 1,092 productos, m = 20 búsquedas por repetición y j = 5 repeticiones. Los valores son aproximados ya que dependen de la carga del sistema en el momento de la medición, pero son coherentes con las complejidades teóricas esperadas.

```
Numero de busquedas por repeticion (m): 20
Numero de repeticiones (j): 5

------------------------------------------------------------

--- Comparacion Lista Enlazada - Arbol AVL ---
 m = 20 busquedas por repeticion | j = 5 repeticiones

------------------------------------------------------------

--- Busquedas exitosas - 20 nombres reales ---
------------------------------------------------------------

  Tiempo total Lista (us): 1608
  Tiempo total AVL (us): 59
  Promedio por busqueda Lista: 16.0800 us
  Promedio por busqueda AVL: 0.5900 us

--- Busquedas fallidas - nombre inexistente ---
------------------------------------------------------------

  Tiempo total Lista (us): 142
  Tiempo total AVL (us): 1
  Promedio por busqueda Lista: 28.4000 us
  Promedio por busqueda AVL: 0.2000 us

--- Totales ---
------------------------------------------------------------

  Lista Enlazada (us): 1750
  Arbol AVL (us): 60
  Resultado: AVL es mas rapido
Factor de mejora AVL - Lista exitosa: x27.25

 --- Complejidades teoricas ---
------------------------------------------------------------

  Lista Enlazada: O(n)
  Arbol AVL: O(log n)
------------------------------------------------------------
```

5. Conclusiones

1) El proyecto demostró que elegir la estructura de datos correcta mejora muchísimo la velocidad del sistema, por ejemplo, usar un árbol AVL fue hasta por mucho más rápido para buscar productos que una lista común, y esta ventaja crece entre más grande sea el catálogo. La clave del éxito fue asignar la mejor herramienta para cada tarea: listas simples para guardar datos rápidamente, árboles AVL para buscar por nombre, árboles B para filtrar por fechas y árboles B+ para agrupar por categorías.

2) Crear estas herramientas desde cero nos sirvió para entender realmente cómo funcionan por dentro, cómo resolver problemas reales y por qué las soluciones profesionales son tan eficientes. El mayor reto fue lograr que al agregar un producto, este se guardara correctamente en todas partes al mismo tiempo, de modo que si algo fallaba, se pudiera "deshacer" el proceso para no arruinar los datos, aunque fue de forma parcial.

3) Los resultados confirman que la teoría es cierta, pero en la vida real la velocidad también depende de cómo la computadora organiza su memoria. Por ejemplo, las listas son más lentas porque sus pedacitos de información quedan esparcidos por todas partes, mientras que los árboles AVL ganan porque necesitan revisar muchísimos menos datos.

4) El sistema utiliza un intermediario para que el programa principal sea muy fácil de usar y no tenga que lidiar con la complejidad interna de las herramientas, esto permite cambiar o probar partes del código sin que lo demás se rompa. Además, para no desperdiciar memoria ni causar fallos en la computadora, se decidió que sólo la lista simple guarde la información real y se encargue de borrarla al final, mientras que los árboles solo guardan accesos directos a esos mismos datos..

5) El proyecto demostró que no existe una herramienta perfecta para todo. Cada una brilla en su propia especialidad: las listas son rápidas para agregar datos nuevos, los árboles AVL son ideales para buscar información puntual, los árboles B son los mejores para filtrar por fechas y los B+ para revisar categorías completas. Un buen sistema combina todas estas opciones y, aunque da más de trabajo mantenerlas siempre coordinadas, la gran recompensa es que el programa responderá a la máxima velocidad sin importar lo que el usuario le pida.